

Localization of Intermittent Hardware Trojans on NoC-based Manycores

Marcos Vinicius Zanardi*, Rafael Follmann Faccenda[‡], Luciano Lores Caimi[†], Fernando Gehm Moraes*

*School of Technology, Pontifical Catholic University of Rio Grande do Sul – PUCRS – Porto Alegre, Brazil

marcos.zanardi@edu.pucrs.br, fernando.moraes@pucrs.br

[‡]Federal University of Minas Gerais – UFMG – Belo Horizonte, Brazil – rafael.faccenda@dcc.ufmg.br

[†]Federal University of Fronteira Sul – UFFS – Chapecó, Brazil – lcaimi@uffs.edu.br

Abstract—Third-party intellectual property in NoC-based manycore systems increases the risk of Hardware Trojan (HT) insertion. This work addresses the runtime localization of intermittent HTs on NoC links, whose irregular activation makes detection particularly challenging. It proposes an HT localization flow for untrusted NoCs, based on traffic monitoring, a NoC Health Map (NHM), and probe-based diagnosis. The NHM accumulates suspicious communication events and triggers localization only when link scores exceed a threshold. To reduce interference with running applications, the flow incorporates two pre-search protocols, Freeze Application and Task Migration, and applies three localization methods: Binary Search Algorithm (BSA), Ordered Search Algorithm (OSA), and Score-Based Search (SBS). BSA+OSA with Freeze achieves the highest localization rate, whereas SBS with Migration provides the best trade-off between localization effectiveness and execution-time overhead. Despite intermittent HT activity and packet losses, applications recover and complete correctly.

Index Terms—NoC-based manycores, security, hardware Trojan (HT), HT localization.

I. INTRODUCTION AND RELATED WORK

Manycore System-on-Chips are widely adopted in computation-intensive domains because they exploit parallel processing to improve performance [1]–[3]. These systems integrate Processing Elements (PEs), which are typically processors, accelerators, memories, and peripherals, interconnected by a Network-on-Chip (NoC) [4]. However, the use of third-party IPs also introduces security risks, including Hardware Trojans (HTs), which may disrupt functionality or leak sensitive information [5]–[7].

Recent work has investigated HTs in NoC-based manycores and proposed several detection and mitigation strategies. Rajan et al. [8] introduced LOKI, a packet-duplication HT inserted in the network adapter, and proposed HULK to suppress repeated malicious requests. In earlier work, the same authors presented a packet-blocking HT targeting router arbitration and proposed the Traffic Snoop Manager to detect abnormal delays and reroute packets around compromised routers [9]. Wang et al. [10, 11] addressed packet-tampering HTs using machine-learning-based anomaly detection and router trust scores, later extending the approach with distributed monitors to improve localization. Dhanuka et al. [12] considered intermittent Black Hole Router attacks and proposed anomaly tables at the network interfaces combined with Dynamic-XY rerouting. Similarly, Sankar et al. [13] used retransmission-based trust tables to identify routers affected by packet corruption. Amara et al. [14] analyzed packet-corruption HTs in compressed-data applications and mitigated them through additional packet-level error-correction support.

Overall, prior work shows that HT localization is often based on distributed data structures that record communication effects and infer suspicious routers or paths. However, these solutions usually place the additional security logic inside the NoC, which is itself part of the untrusted infrastructure, and often incur non-negligible hardware overhead due to tables or ML-based decision modules.

To address these limitations, we propose an HT localization flow that injects batches of probe packets into selected NoC links to investigate malicious behavior, including intermittent attacks. A dedicated Manager PE (MPE) coordinates the flow using a NoC Health Map (NHM) updated by platform monitoring mechanisms. We also introduce two techniques to reduce the impact of probe traffic on both the NoC and the running applications. The main contributions are: (i) an NHM that stores suspicious events observed on NoC links; (ii) two probe-batch-based HT localization methods capable of identifying links affected by intermittent HTs; (iii) a security-driven task migration algorithm to relocate tasks away from compromised regions; and (iv) an evaluation of the techniques in terms of execution time overhead and HT localization rate.

II. THREAT MODEL

Practical HTs often employ obfuscation mechanisms that obstruct detection. Intermittent HTs are particularly challenging because they activate briefly, disrupt system operation, and then return to a dormant state, reducing the probability of successful localization [15].

In this work, the data NoC may contain intermittent time-triggered HTs inserted in any router link. The MPE (including the NHM), and control NoC are assumed to be trusted. Each HT uses a Finite State Machine (FSM) to define active and idle intervals, using a Linear Feedback Shift Register (LFSR) outputs to make them variable. During activation, the HT masks the router’s transmission signal, preventing the receiving router from detecting the transfer and resulting in partial or complete packet loss.

Figure 1 illustrates the behavior of an HT in an infected NoC link. Green bars denote HT activation intervals, whereas purple bars represent probe packets traversing the link. The figure shows four HT activations with distinct active and idle durations. In two of them, packets 15, 16, 25, and 26 are lost (red arrows), indicating that they do not reach the destination and therefore trigger a timeout. This example illustrates the main challenge in HT localization: detection depends on probe packets overlapping an HT activation interval. Therefore, practical methods cannot guarantee the detection of all HTs.

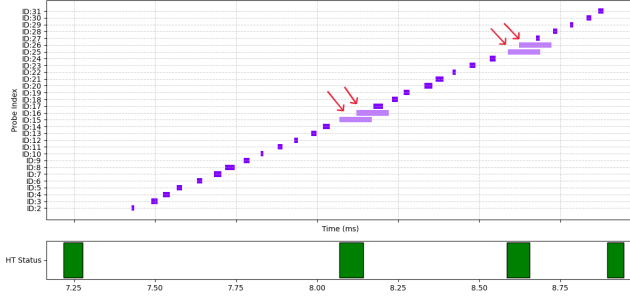


Fig. 1: Example of intermittent HT acting over a NoC link.

III. HT LOCALIZATION FLOW

Figure 2 presents the proposed HT localization flow. Applications are modeled as Communicating Task Graphs (CTGs), in which vertices represent tasks and edges represent communications between tasks mapped to different PEs.

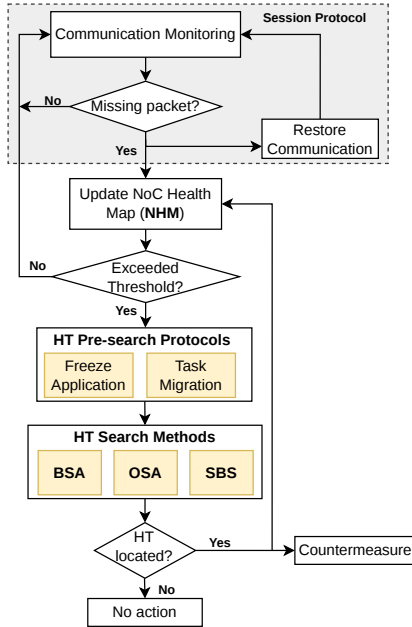


Fig. 2: HT Localization Flow.

Communication monitoring relies on the **session protocol** [16], which detects packet loss and restores communication when required. The platform employs two NoCs: a data NoC [17] and a control NoC [18]. The data NoC uses wormhole switching and supports XY and source routing, whereas the control NoC uses broadcast communication to ensure packet delivery even in the presence of HTs or faults. For each inter-PE communication, the source task sends a data packet through the data NoC and a control packet through the control NoC. If the target PE does not receive both packets within a predefined interval, it reports a missing-packet event, and the session protocol restores communication through an alternative source-routed path.

HT localization is supported by the NoC Health Map (NHM), which is updated by the MPE at software level. Missing-packet events generated by the session protocol in-

crease the link scores along the corresponding source–target path. Because missing-packet events may also result from transient congestion, HT localization is triggered only when at least one link score exceeds a predefined threshold.

Before localization, a **pre-search protocol** prevents application traffic from traversing suspicious links. The two options are Freeze Application and Task Migration.

Once suspicious links are isolated from application traffic, the localization phase begins. The flow integrates one prior localization method (**BSA**) [19] with two methods proposed in this work: **OSA** and **SBS**. All methods use probe packets injected to reveal HT activity when they traverse a link during an HT activation interval.

If a method identifies an HT, the corresponding NHM entry is marked as *infected*. Since HTs may be intermittent, localization may fail even when a link is compromised. In this case, no immediate action is taken, and the search is repeated after a new event on that link.

HT localization can support countermeasures, although they are outside the scope of this work. Possible actions include isolating infected links from data traffic and constraining task mapping away from compromised regions.

IV. NHM AND PRE-SEARCH PROTOCOLS

The NoC Health Map (**NHM**) is a centralized data structure that stores, at software level, monitoring information produced by the session protocol and the localization mechanisms. It provides a global view of NoC link conditions, distinguishing regions with potential HT activity.

The NHM is implemented as a three-dimensional matrix. The first two dimensions correspond to the router’s coordinates (according to NoC topology), while the third represents the router’s output ports (e.g., East, West, North, and South). Each entry corresponds to one NoC link and stores three attributes:

- **Health Status:** classifies the link as *healthy*, *suspicious*, or *infected*;
- **Suspicion Score:** updated by session protocol, stores the number of interrupted communications on that link;
- **Probe Counters:** the number of transmitted probe packets and unsuccessful probes.

Figure 3a shows a manycore containing four HTs. In Figure 3b, two eastbound links exceed the predefined suspicion threshold (set to 3), due to repeated failures in the communications $D0 \rightarrow DIV$, $D1 \rightarrow DIV$, and $D2 \rightarrow DIV$ of the green application. Together, these attributes allow the NHM to support both HT localization and subsequent countermeasures. Even when the exact HT location cannot be determined, the NHM still indicates suspicious regions.

A. Pre-search Protocols

The first pre-search protocol is **Freeze Application**, illustrated in Figure 4. It suspends applications that use suspicious links, preventing application traffic from interfering with HT localization.

When a communication failure occurs, the target PE notifies the MPE, which requests the corresponding path from the source PE (Path Source PE in Figure 4). The MPE then consults the NHM to identify the PEs along this path that are connected by links with suspicion scores exceeding a

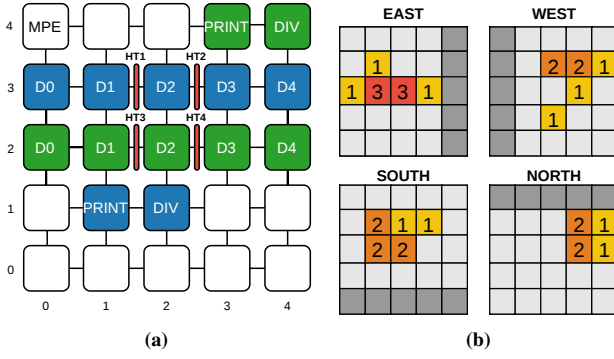


Fig. 3: Manycore scenario with HTs (a), running two applications (colored in green and blue). (b) NHM suspicion scores.

predefined threshold. Next, it broadcasts a freeze command containing the identifiers of the affected applications. Any PE executing a task from one of these applications blocks its scheduler, suspending task execution and communication. Once all affected tasks are blocked, the application is considered frozen. The buffers of the routers along the suspicious path are then reset to remove residual packets and provide a clean state for the subsequent packet exchanges.

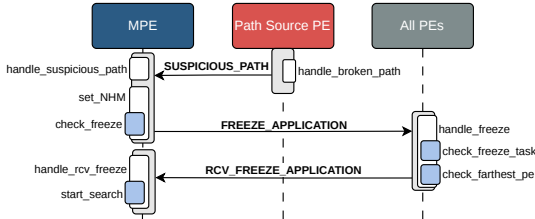


Fig. 4: Freeze Application Protocol.

The second pre-search protocol is **Task Migration**, illustrated in Figure 5. This protocol relocates tasks from suspected regions to safer areas, removing application traffic from the suspect links. When a communication failure is reported by the Path Source PE, the MPE analyzes the NHM scores for the source-target path that triggered the event. Any task mapped to a PE connected to a link in this path, with a score above the threshold, is added to the migration list. Even in the presence of HT activity, the rerouting mechanism ensures correct task migration by redirecting migration traffic through an alternative healthy path whenever packet loss is detected.

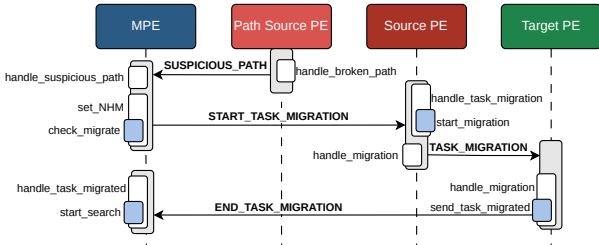


Fig. 5: Task Migration Protocol.

To select a target PE for migration, the MPE applies a table-based algorithm that consults the NHM. The algorithm searches for the closest PE to the Source PE, avoiding links

with high suspicion scores. The selected PE is the one whose links have the lowest suspicion scores.

After selecting the target PE to receive the task, the MPE sends a migration command to the Source PE through the control NoC. The task is then transferred from the Source PE to the Target PE via the data NoC. This transfer continues until the entire task context and memory contents have been relocated. The MPE can execute the migration protocol for multiple tasks simultaneously. After all tasks in the migration list have been moved, the suspected region has no more application traffic, allowing the HT search methods to proceed without interfering with active tasks.

For example, Figure 3b shows the scores of all PEs in a manycore scenario. PEs (1,2) and (2,2) have scores exceeding the threshold, set to 3, and, therefore, are considered suspicious. Upon consulting the NHM, the MPE identifies PEs (1,4) and (2,4) as secure places to relocate the affected tasks, as they have a score of 0 on every link, being considered healthy links.

V. HT LOCALIZATION METHODS

Binary Search Algorithm (BSA) [19] — An end-to-end method for localizing an HT that follows a divide-and-conquer strategy. The MPE partitions the path into two sub-paths and sends probe packets through both in parallel. Segments with successful probe delivery are discarded, while those associated with probe failure are recursively subdivided. The procedure continues until the suspicious segment is reduced to a single hop, which identifies the infected link. The MPE manages this process using a Binary Search Table that maps each probe identifier to its corresponding path. When a segment is split, new entries are created for the resulting sub-paths. Upon receiving a probe result, the MPE updates the search state and removes or replaces table entries accordingly.

Ordered Search Algorithm (OSA) — A hop-by-hop method that tests suspicious links individually along a path. Unlike BSA, which recursively partitions the path, OSA examines links sequentially using *probe batches*. When triggered, OSA builds an array containing all links in the suspicious path. For each link, it stores the source router, output port, and NHM suspicion score. The array is sorted in descending order of suspicion; ties are resolved by prioritizing links closer to the target PE. OSA then launches one probe batch per link and waits for the result. If all probes in a batch succeed, the OSA advances to test the next candidate link. If at least one probe fails, the link is marked as *infected*, and the search stops. OSA terminates upon finding an *infected* link or after evaluating all candidates.

Score-Based Search (SBS) — A hop-by-hop method that probes only links whose suspicion scores exceed a predefined threshold. In contrast to OSA, which analyzes suspicious paths, SBS focuses directly on the most suspicious links in the NHM. When triggered, the MPE consults the NHM and records all links whose scores exceed the threshold in a probing array. Probe batches are then launched only through these links. By restricting the search space, SBS avoids unnecessary probes on low-risk links and reduces the impact of localization on unaffected NoC regions. Consequently, SBS concentrates probing overhead on the most suspicious areas of the data NoC, enabling faster and more efficient searches.

TABLE I: Execution time overhead for the simulated scenarios, combining the pre- and search methods.

Scenario	Baseline (ms)	BSA + OSA				SBS			
		Freeze (ms)	Overhead (%)	Migration (ms)	Overhead (%)	Freeze (ms)	Overhead (%)	Migration (ms)	Overhead (%)
I	14.88	21.99	47.78	20.29	36.36	22.02	47.98	21.67	45.63
II	18.18	24.85	36.69	20.08	10.45	20.00	10.01	19.70	8.36
III	15.09	19.33	28.10	17.57	16.43	18.92	25.38	18.44	22.20
IV	18.45	23.00	24.66	19.52	5.80	21.07	14.20	19.98	8.29
V	33.24	43.24	30.08	34.75	4.54	40.22	21.00	34.23	2.98
Geo. Mean	-	-	32.50	-	11.04	-	20.53	-	11.59

TABLE II: Number of found HTs for the simulated scenarios, combining the pre- and search methods.

Scenario	BSA + OSA				SBS			
	Freeze		Migration		Freeze		Migration	
	Attempts	Found HT	Attempts	Found HT	Attempts	Found HT	Attempts	Found HT
I	6	HT1	4	HT1, HT2	6	None	4	HT1
II	10	HT2	3	HT2	4	None	6	HT2
III	4	HT1, HT2	6	HT1	4	HT2	4	HT1, HT2
IV	7	HT3	10	None	6	HT1	6	None
V	20	HT1, HT3, HT4	10	HT3	15	HT1, HT2, HT4	7	HT4
Total	47	8 (50%)	33	5 (31.25%)	35	5 (31.25%)	27	5 (31.25%)

VI. RESULTS

This section evaluates the proposed flow in terms of execution-time overhead and HT localization capability. Results are obtained from cycle-accurate RTL simulation [20]. Five simulation scenarios are considered (Figure 6 illustrates two of the evaluated scenarios): (I) one Dijkstra application, (II) one DTW, (III) two Dijkstra, (IV) two DTW, and (V) DTW with MPEG. For each scenario, a no-attack baseline simulation establishes the reference execution time. Scenarios with two applications include four HTs, whereas single-application scenarios include two HTs. The HT placement is based on high-traffic links to increase the probability of packet-HT collisions.

The evaluation compares two search approaches: path-based and link-based. The path-based approach combines **BSA** and **OSA**: BSA is executed first; if it does not locate an HT, OSA then performs a hop-by-hop search along the same path. The link-based approach relies exclusively on **SBS**. In both cases, one pre-search protocol is applied before localization. All experiments use probe batches of 10 packets with 40 flits each and an inter-batch interval of 100 μ s.

Table I presents the execution time and overhead results for each scenario. The overhead is computed relative to the baseline execution time. The last row summarizes the geometric mean of the overhead values for each combination of pre-search and search method. The results show that the **Migration** pre-search protocol consistently presents lower execution-time overhead. Its cost is limited to relocating tasks away from the region under analysis, after which the application resumes execution while HT localization proceeds in parallel. In contrast, **Freeze** incurs a higher penalty because the application remains stalled during HT localization. The geometric mean shows that **Migration** reduces the overhead from 32.5% to 11.04% in the BSA+OSA case, and from 20.53% to 11.59% in the SBS case. The reduction is more pronounced for BSA+OSA because this strategy evaluates every link along the path, including those with low suspicion scores, thereby increasing localization time and, consequently,

the duration the application remains frozen.

Table II reports the number of probed links (*attempts*) and the number of located HTs. The last row summarizes the total number of links searched and the percentage of HTs located out of 16. The results show that the combination of BSA+OSA with **Freeze** performs the highest number of searches, 47 links. This configuration also achieves the highest detection rate, locating 8 out of 16 HTs (50%). The remaining configurations locate 5 out of 16 HTs each (31.25%). Although this rate may appear low, it is relevant because intermittent HTs may evade some diagnostic rounds, but the protocol continues probing across subsequent application iterations, updating the NHM and triggering new searches.

SBS with **Migration** locates the same number of HTs as SBS with **Freeze** and BSA+OSA with **Migration**, while requiring fewer searches (27), which suggests higher search efficiency. Overall, the results reveal a clear trade-off between localization intensity (*attempts*) and runtime overhead, while confirming that, even in the presence of HTs and packet losses, the proposed flow enables the applications to recover and complete execution correctly.

VII. CONCLUSIONS AND FUTURE WORK

The proposed flow provides an effective trade-off between HT localization capability and runtime overhead. BSA+OSA with **Freeze** achieves the highest localization rate, whereas SBS with **Migration** yields the lowest overhead. Most importantly, even under HT activity and packet losses, applications recover and complete execution correctly. In addition, the proposed localization flow can be extended beyond HTs, which may cause packet loss, if the platform includes mechanisms to detect and recover from other types of HT attacks. Thus, the same framework can be applied to other threats, such as packet corruption, provided that corresponding detection and correction mechanisms are available. Future work will address probe-parameter optimization, broader long-term evaluation, and HT-aware countermeasures such as secure remapping and infected-link isolation.

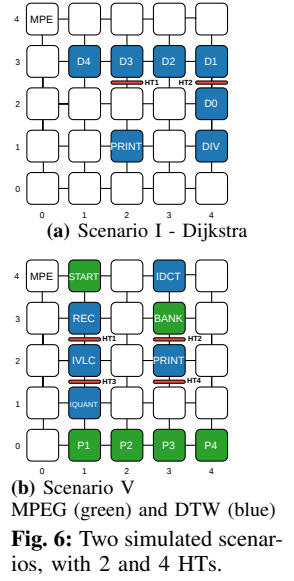


Fig. 6: Two simulated scenarios, with 2 and 4 HTs.

ACKNOWLEDGMENTS

This work was financed in part by Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), 305621/2024-6; Fundação de Amparo à Pesquisa do Estado do Rio Grande do Sul (FAPERGS), 23/2551-0002200-1.

REFERENCES

- [1] J. Haase, N. Volkens, and D. Goehringer, "Embedded Security Accelerators within Network-on-Chip Environments," in *HEART*, 2024, pp. 37–43, <https://doi.org/10.1145/3665283.3665290>.
- [2] S. Hazra, B. Avinash, and M. Dalui, "Design, threat analysis and countermeasures for cache replacement policy-affecting Hardware Trojans in the context of a many-core system," *Microelectronics Journal*, vol. 142, p. 105973, 2023, <https://doi.org/10.1016/j.mejo.2023.105973>.
- [3] M. Kumar, A. K. Swain, K. Mahapatra, and S. P. Mohanty, "Fortified-NoC: Robust Approach for Trojan-Resilient Network-on-Chips to Fortify Multicore-Based Consumer Electronics," *IEEE Transactions on Consumer Electronics*, vol. 68, no. 1, pp. 57–68, 2022, <https://doi.org/10.1109/TCE.2021.3129155>.
- [4] M. Hashemi, S. Mohammadi, and T. E. Carlson, "TOP: A Combined Logical and Physical Obfuscation Method for Securing Networks-on-Chip Against Reverse Engineering Attacks," *IEEE Access*, vol. 13, pp. 133 438–133 456, 2025, <https://doi.org/10.1109/ACCESS.2025.3590967>.
- [5] S. Charles, Y. Lyu, and P. Mishra, "Real-Time Detection and Localization of Distributed DoS Attacks in NoC-Based SoCs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 12, pp. 4510–4523, 2020, <https://doi.org/10.1109/TCAD.2020.2972524>.
- [6] B. Bisht and S. Das, "BHT-NoC: Blaming Hardware Trojans in NoC Routers," *IEEE Design & Test*, vol. 39, no. 6, pp. 39–47, 2022, <https://doi.org/10.1109/MDAT.2022.3202998>.
- [7] M. M. Ahmed, A. Dhavle, N. Mansoor, S. M. P. Dinakarrao, K. Basu, and A. Ganguly, "What Can a Remote Access Hardware Trojan do to a Network-on-Chip?" in *ISCAS*, 2021, pp. 1–5, <https://doi.org/10.1109/ISCAS51556.2021.9401297>.
- [8] M. Rajan, A. Das, and J. Jose, "Securing Network-on-Chips against Trojan-Induced Packet Duplication Attacks," *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 4, 2025, <https://doi.org/10.1145/3744645>.
- [9] M. Rajan, A. Phadke, S. Sankar, and J. Jose, "Handling Packet Blocking Attack in NoC Using Traffic Snooping," in *COINS*, 2024, pp. 1–6, <https://doi.org/10.1109/COINS61597.2024.10622599>.
- [10] H. Wang and B. Halak, "Hardware Trojan Detection and High-Precision Localization in NoC-Based MPSoC Using Machine Learning," in *ASP-DAC*, 2023, pp. 516–521, <https://doi.org/10.1145/3566097.3567922>.
- [11] —, "TampML: Tampering Attack Detection and Malicious Nodes Localization in NoC-Based MPSoC," *IEEE Transactions on Emerging Topics in Computing*, vol. 13, no. 2, pp. 551–562, 2025, <https://doi.org/10.1109/TETC.2024.3434663>.
- [12] G. Dhanuka, S. Sankar, and J. Jose, "Dynamic Mitigation of Hardware Trojan Induced Black Hole Router Attack in Network-on-Chip," in *ISVLSI*, 2025, pp. 1–6, <https://doi.org/10.1109/ISVLSI65124.2025.11130261>.
- [13] S. Sankar, R. Gupta, J. Jose, and S. Nandi, "TROP: TRust-aware Opportunistic Routing in NoC with Hardware Trojans," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 2, 2024, <https://doi.org/10.1145/3639821>.
- [14] H. Amara, C. Killian, D. Chillet, and E. Casseau, "Mitigation of Hardware Trojan in NoC using Delta-Based Compression," in *SOCC*, 2024, pp. 1–6, <https://doi.org/10.1109/SOCC62300.2024.10737773>.
- [15] J. Philomina, "A Study on the Effect of Hardware Trojans in the Performance of Network on Chip Architectures," in *International Conference on Smart Computing and Communications (ICSCC)*, 2021, pp. 314–318, <https://doi.org/10.1109/icsc51209.2021.9528249>.
- [16] R. F. Faccenda, G. Comarú, L. L. Caimi, and F. G. Moraes, "A Comprehensive Framework for Systemic Security Management in NoC-Based Many-Cores," *IEEE Access*, vol. 11, pp. 131 836–131 847, 2023, <https://doi.org/10.1109/ACCESS.2023.3336565>.
- [17] F. G. Moraes, N. Calazans, A. Mello, L. Moller, and L. Ost, "HERMES: an infrastructure for low area overhead packet-switching networks on chip," *Integration the VLSI Journal*, vol. 38, no. 1, pp. 69–93, 2004, <https://doi.org/10.1016/j.vlsi.2004.03.003>.
- [18] E. Wachter, L. L. Caimi, V. Fochi, D. Munhoz, and F. G. Moraes, "BrNoC: A broadcast NoC for control messages in many-core systems," *Microelectronics Journal*, vol. 68, pp. 69–77, 2017.
- [19] G. Comarú, R. F. Faccenda, L. L. Caimi, and F. G. Moraes, "Hardware Trojan Localization for Untrusted Network-on-Chips," in *LASCAS*, 2025, pp. 1–5, <https://doi.org/10.1109/LASCAS64004.2025.10966274>.
- [20] M. Ruaro, L. L. Caimi, V. Fochi, and F. G. Moraes, "Memphis: A Framework for Heterogeneous Many-core SoCs Generation and Validation," *Springer Design Automation for Embedded Systems*, vol. 23, no. 3–4, pp. 103–122, 2019, <https://doi.org/10.1007/s10617-019-09223-4>.